HireLLMe

Technical Documentation

LLM-powered job triage pipeline with Gmail, Gupy, Python and GitHub Actions

Kevin Mota da Costa

Portfolio: costakevinn.github.io LinkedIn: linkedin.com/in/costakevinnn

November 9, 2026

Abstract

HireLLMe turns the job alerts a candidate receives every day into a short, organised list of openings worth applying to. It reads LinkedIn and Indeed alerts from Gmail over IMAP, searches the Gupy job portal, stores every posting in an append-only raw layer, deduplicates them within and across sources, asks a large language model to triage each new opening against a written candidate profile, and delivers one email per job into Gmail labels, naming the resume to use. The pipeline runs twice a day on GitHub Actions, works with any LLM provider (a Claude subscription, the Anthropic API or any OpenAI-compatible API) and is designed so that no job is ever lost, evaluated twice or delivered twice. Applying remains a human decision: the pipeline never applies. This document describes the sources, the data layers, the LLM triage and its failure handling, the delivery, the orchestration, the security model, the results and the limitations.

Contents

1 Introduction **3**

2 Architecture **3**

3 Sources **4**

3.1 Gmail as a queue 4

3.2 LinkedIn 5

3.3 Indeed 5

3.4 Gupy 6

3.5 Jobs sent by the candidate 6

4 Data Layers **6**

4.1 Bronze: raw layer 6

4.2 Silver: clean table 7

4.3 Gold: triaged table 7

5 LLM Triage **7**

5.1 Request . . . 7
5.2 Output contract . . . 7
5.3 Providers . . . 8
5.4 Error taxonomy . . . 8
5.5 Queue policy and budget . . . 9
5.6 Jobs sent by the candidate . . . 9

6 Delivery **9**

7 Orchestration **10**
7.1 Runtime . . . 10
7.2 Scheduled runs . . . 10
7.3 Notices . . . 10

8 Reliability **10**

9 Security and Privacy **11**

10 Results **12**
10.1 First run . . . 12
10.2 Daily runs . . . 12

11 Testing **12**

12 Reproducibility **13**

13 Limitations and Future Work **13**

1 Introduction

A candidate who sets up job alerts on several platforms soon receives dozens of emails a day. The same opening arrives from more than one source, most postings do not match the candidate's level or location, and the links in the emails are long tracking URLs, some of them carrying login tokens. Reading all of it by hand is slow, and the good openings are easy to miss.

HireLLMe automates the reading and the first filtering, and leaves the decision to apply to the person. It was built around five requirements:

- **Nothing is lost.** An alert that cannot be read now is retried later or set aside for a human; a job that cannot be triaged now waits in a queue that never expires.
- **Nothing is done twice.** Each email is read once, each job is evaluated by the LLM once and delivered once, even when a run stops halfway and starts again.
- **Provider independence.** The triage works with a subscription plan, a paid API or a free tier, and reacts to every provider failure in the same way.
- **Privacy.** Credentials and personal data never reach the public repository, and login tokens found in alert links are never stored.
- **No automatic applications.** The output is an organised inbox; every application and every resume choice is made by the candidate.

The code is written in Python with the standard library only (about 1,700 lines in twelve modules, plus 19 unit tests), runs in Docker, and is scheduled on GitHub Actions.

2 Architecture

The pipeline has four steps, each a separate program that reads the output of the previous one and works only on what is still pending (Figure 1). A step that fails does not prevent the next ones from running, and whatever it left undone is picked up by the next run.

Module	Role
`collect.py`	Step 1: reads the Gmail queue and searches Gupy; writes the raw layer
`clean.py`	Step 2: rebuilds the clean table from the raw layer
`triage.py`	Step 3: sends pending jobs to the LLM and validates the answers
`deliver.py`	Step 4: creates one email per job in the Gmail labels
`pipeline.py`	Runs the four steps, sends notices, sets the exit code
`sources.py`	Parsers: LinkedIn, Indeed, Gupy and links sent by the candidate
`llm.py`	One interface to every LLM provider, with a common error taxonomy
`gmail.py`	IMAP access: read a label, move messages, create messages
`notify.py`	Notice emails: missed or failed runs, triage problems, token expiry
`common.py`	Paths, configuration, UTC time, JSONL and CSV helpers
`purge.py`, `reset.py`	Maintenance tools (dry run unless `--yes`)

Table 1: Modules in `scripts/`.

Every user-facing setting lives in `data/config.json`: the sources that are on, the Gmail labels, the search terms, the LLM provider and model, the budget, and every text of the delivered

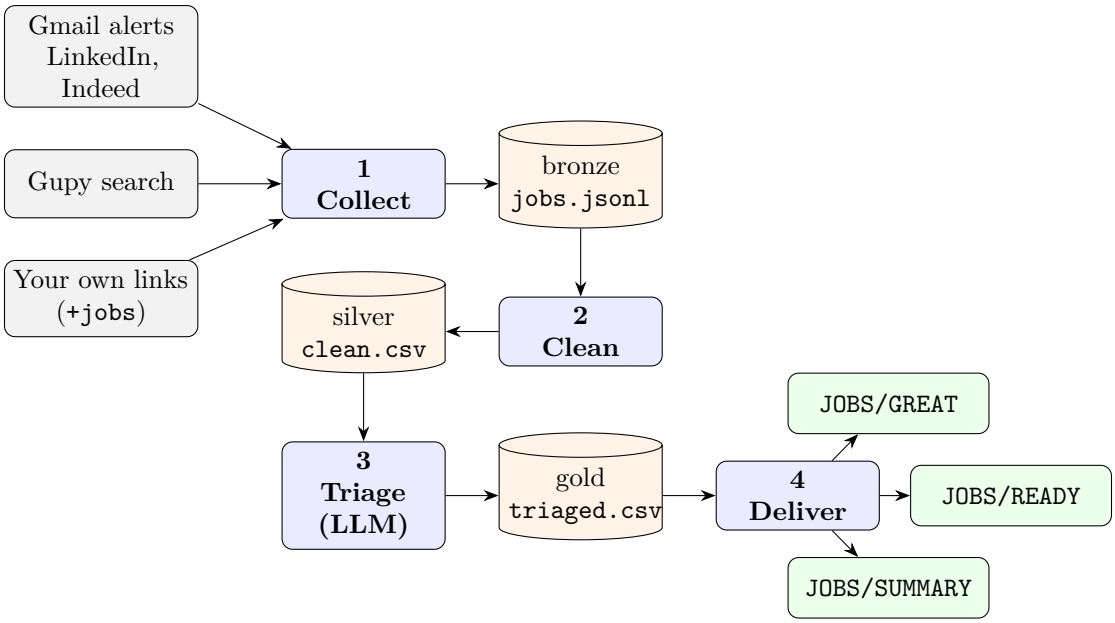


Figure 1: The four steps and the three data layers. Grey: sources; blue: steps; orange: files; green: Gmail labels.

emails. The candidate profile lives in `data/profile.md`. Both are personal files, ignored by Git in the public repository, each with a `.example` file next to it.

3 Sources

3.1 Gmail as a queue

Alerts reach a Gmail label (`JOBS` by default) through Gmail filters, one per alert sender. The label is the queue of unprocessed alerts: an email leaves it once it has been handled, so each run simply reads what is left. Access is over IMAP [1] with an app password; reading uses `BODY.PEEK[]`, so it never changes the state of a message, and all recent messages are fetched in one request.

After reading, each email is moved according to the outcome (Table 2). A move is a `COPY` to the target label followed by `STORE \Deleted` and `EXPUNGE` in the source label, which in Gmail only removes the source label.

Outcome	Action
Jobs read, or the email was already read (same `Message-ID`)	Moved to `JOBS/PROCESSED`, marked read
Source turned off in the configuration	Moved to `JOBS/PROCESSED`, marked read
Transient error (a link did not answer)	Stays in `JOBS`; retried on the next run
still failing after `days_until_manual` days	Moved to `JOBS/MANUAL`, unread, keeping the jobs that could be read
Permanent error (unknown sender or format, no job found)	Moved to `JOBS/MANUAL`, unread

Table 2: What happens to each email in the queue.

Order of operations. A run first appends the jobs to the raw layer, then records the run, and only then moves the emails. If it stops between the first and the last operation, the emails are still in the queue and are read again, but their `Message-ID` is already in the raw layer, so they are recognised and only moved. No job is lost and none is duplicated.

First run. An inbox may hold weeks of old alerts. On the first run (when the run history is empty) only the last `first_run_days` days are read (14 by default); older alerts are moved to `JOBS/PROCESSED` without being downloaded. Later runs read the whole queue, whatever the age of each message, so a skipped run never causes a job to be missed.

Without filters. Gmail filters cannot be created from the mobile app. As an alternative, the queue can be the inbox itself, restricted on the server to a list of senders with an IMAP `SEARCH` of the form `(ALL) (OR FROM a OR FROM b FROM c)`; moving a message out of the inbox then works like archiving it. Reading the whole inbox without a sender list is refused.

3.2 LinkedIn

LinkedIn alerts are parsed from their plain-text part, a sequence of blocks with the title, the company, the location and a link to the job. The link in the email carries a one-time login token (`otpToken`) that would open the candidate's session; it is never stored. Only the numeric job id is extracted, and the link is rebuilt as `https://www.linkedin.com/jobs/view/<id>`. LinkedIn itself is never accessed.

3.3 Indeed

Indeed sends two kinds of alerts: a list of jobs, and a single job "matched" to the candidate. Each job is identified by a 16-digit hexadecimal key, `jk`, and its clean link is `https://<domain>/viewjob?jk=<jk>`, where the domain is the Indeed site of the candidate's country. The links in the emails are tracking links whose format has changed over time:

- **Links with the key in the query** (`/rc/clk/dl?jk=...`): the key is read directly, with no request.
- **Compressed links** (`cts.indeed.com/v3/<payload>`): the payload is the destination URL compressed with gzip [2] and encoded in URL-safe base64 [3], so the key is decoded offline:

```
payload = gzip.decompress(base64.urlsafe_b64decode(token + "=" * (-len(token) %
    4)))
jk = RE_JK.search(payload.decode("utf-8", "replace")).group(1)
```

- **Redirect links** (`engage.indeed.com/f/a/...`): the redirects are followed one hop at a time, without following the last one, until a URL with the key appears. The job page itself is never opened. Up to ten links are followed in parallel.
- **Sponsored jobs** (`/pagead/clk/...`): they carry no key, and Indeed answers any automated request with HTTP 403. The email link is kept as it is: it still works in a browser, and the cleaning step can still merge the job by company and title.

A link that does not answer is a transient error (Table 2). Following redirects can be turned off in the configuration, in which case every email link is kept as it is.

3.4 Gupy

Gupy, a recruitment platform widely used in Brazil, does not send alerts; it is searched through the endpoint used by its public job portal, `portal.gupy.io/api/job-search/jobs`, once per configured search term, 100 results per page, with a one-second pause between requests. Only postings published since the start of the last run in which the search succeeded (minus a one-hour overlap) are kept, so a failed search is simply repeated over a longer window by the next run. Unlike the email alerts, Gupy returns the full description, the work model (remote, hybrid, on-site) and the application deadline.

3.5 Jobs sent by the candidate

The candidate can add a job found elsewhere by emailing its link, from their own account, to a `+jobs` address that a Gmail filter places in the queue. Every link in the body becomes a job. The subject is parsed in two passes: tags select the label and the resume, and the rest is split on `·` or `|` into title, company and location.

Subject	Parsed as
`Data Scientist · Acme · Remote`	title, company, location; no tag
`#great Data Scientist · Acme`	label *great*; title, company
`#ready #data Data Engineer`	label *normal*, resume `data`; title

Table 3: Subjects of jobs sent by the candidate. The tag words are set in the configuration (for example `#otima` in Portuguese).

How these jobs are triaged is described in Section 5.6.

4 Data Layers

The data follows a bronze, silver and gold layout. Each layer is a plain file, kept in the private repository and versioned by Git.

4.1 Bronze: raw layer

`raw/jobs.jsonl` receives one JSON line per job found, and lines are only ever appended. Each record keeps the source, the clean link, the parsed fields, the description when there is one, the collection time and the origin:

```
{"source": "indeed", "url": "https://br.indeed.com/viewjob?jk=0123456789abcdef",
 "title": "Data Engineer", "company": "Acme", "location": "Sao Paulo, SP",
 "description": "...", "collected_at": "2026-11-09T20:55:30Z",
 "origin": {"type": "email", "sender": "donotreply@jobalert.indeed.com",
            "subject": "...", "received": "...", "message_id": "<...>"}}
```

Two logs sit next to it: `raw/runs.jsonl`, one line per collection (time window, emails by outcome, jobs by source, whether Gupy succeeded), and `raw/triage.jsonl`, one line per triage run (provider, model, jobs sent and decided, tokens used, and why the run stopped, if it did). All times are stored in UTC.

4.2 Silver: clean table

`clean.csv` is rebuilt from scratch from the raw layer on every run, so a change in the cleaning rules applies to the whole history at once.

- **One row per link.** The job id is the first eight hexadecimal digits of the SHA-1 of the clean link. It does not depend on the order of collection, so it stays stable across rebuilds and is what links the silver and gold layers.
- **Cross-source merge.** Company and title are normalised (lower case, accents and punctuation removed, a leading "pessoa" dropped, as in "Pessoa Analista de Dados"); rows with the same normalised pair are merged into one, keeping the other links in `other_links` and every source in `sources`.
- **Excluded titles.** A configurable regular expression removes postings that are not for the candidate (for example, positions reserved for people with disabilities).

4.3 Gold: triaged table

`triaged.csv` holds one row per decided job: the decision (`category`, `resume`, `language`, `reason`), a copy of the job fields, the time of the decision and `delivered_at`, empty until the job's email has been created. Pending work is defined by the layers themselves: a job is waiting for triage when its id is in `clean.csv` but not in `triaged.csv`, and waiting for delivery when `delivered_at` is empty. No separate state needs to be kept in sync.

5 LLM Triage

5.1 Request

Pending jobs are sent in batches of up to 40, newest first. Each request has two parts:

- **System prompt** (`prompts/triage.md`): the task, the three categories, the rules ("when in doubt, normal"; never invent a requirement the posting does not state) and the output format.
- **User message**: the resume names and language codes allowed, the language in which to write the reasons, the candidate profile (`profile.md`, which can be written in any language), and the batch as a JSON array with id, title, company, location, sources and up to 1,500 characters of description.

The profile is the filter: it says who the candidate is, which resumes exist and what makes a job great, normal or rejected. Tuning the triage means editing this text, not the code.

5.2 Output contract

The model must answer with a JSON array, one object per job:

```
[{"id": "3f9a1c2e", "category": "great", "resume": "ml", "language": "EN",
  "reason": "Junior ML role, remote, Python and PyTorch match the profile"}]
```

The answer is validated in code: the array is extracted from the text, ids that were not in the batch are ignored, and a decision is accepted only if its category, resume and language belong

to the allowed sets. A job without a valid decision is not written to the gold layer, so it stays pending and is sent again.

5.3 Providers

`llm.py` exposes a single call, `complete(system, prompt, config)`, which returns the text and the token usage, whatever the provider (Table 4). The provider, the model and, when needed, the base URL are set in the configuration; the key is read from the environment.

Provider	How it is called
`claude-code`	Claude Code in headless mode (`claude -p`, no tools, JSON output), authenticated with the token of a Claude Pro or Max subscription. The process refuses to run if `ANTHROPIC_API_KEY` is set, since that would switch it to paid API usage.
`anthropic`	HTTPS request to the Anthropic Messages API with an API key.
`openai-compatible`	HTTPS request to any `/chat/completions` endpoint: OpenAI, Gemini, Groq, OpenRouter or a local Ollama server.

Table 4: LLM providers. The default is Claude Haiku through a Claude subscription.

A subscription to a chat product does not include API access (ChatGPT Plus and the Gemini app, for example); the Claude subscription is usable only because Claude Code accepts its token. Gemini and Groq offer free API tiers, which makes the pipeline usable at no cost.

5.4 Error taxonomy

Every provider fails in its own way: HTTP status codes, error bodies, or the text of a command-line tool. `llm.py` maps each failure, from its status code or its message, to one of four kinds, and each kind has exactly one reaction (Table 5).

Kind	Examples	Reaction
`setup`	Missing, invalid, revoked or expired key; unknown model (401, 403, 404)	Stop the triage and notify the candidate at once
`limit`	Rate limit, plan limit, quota or credit used up (429)	Stop the triage; jobs wait for the next run. Notify only after `alert_after_runs` consecutive runs (default 3)
`transient`	Timeout, server error, overload (5xx)	Retry twice, after 10 and 30 seconds; then treat as `limit`
`output`	No JSON array, invalid JSON	Split the batch in two and ask again; a job that still fails alone is delivered as *not triaged*

Table 5: Error kinds and reactions. The same table applies to every provider.

The `output` reaction is a binary search for the job that breaks the answer: a batch of n jobs that keeps failing costs at most about $2 \log_2 n$ extra requests to isolate one bad job, and the other jobs of the batch are still decided.

5.5 Queue policy and budget

- **Health check.** Before the first batch, the model is asked to reply with one word. If the provider is unavailable, nothing is sent and the run records why it stopped. The same check is available on its own as `triage --check`.
- **No expiry.** A job waits until it is triaged; newer jobs go first, so a backlog never delays today's alerts.
- **Budget.** `max_jobs_per_run` caps the jobs sent per run, for small free tiers; the rest wait.
- **Delivery without a decision.** Optionally (`deliver_untriaged_after_days`), a job that has waited too long is delivered as *not triaged*, without spending tokens, so the candidate still sees it.
- **Accounting.** The tokens of every request are added up and written to `raw/triage.jsonl`; a dry run estimates the input tokens of the pending requests (about four characters per token) without calling the model.

The triage exits with code 3 when the candidate must be told (a `setup` error, or a `limit` that has lasted several runs). The pipeline turns that code into a notice email without marking the run as failed, since nothing was lost.

5.6 Jobs sent by the candidate

A job the candidate sent is a decision already half made, so the model's answer is adjusted by three rules:

1. it is never rejected: without a label tag, the model is told to choose between great and normal, and a rejection is turned into normal;
2. a label tag sets the category, and a resume tag sets the resume; the model still writes the reason and picks the resume when there is no tag;
3. if the provider is unavailable, a job with a label tag is delivered anyway.

These jobs go to the front of the queue, and their reason starts with a configurable prefix ("sent by you"). If the candidate sends a job that was already decided, for example one the filter rejected, the gold row is updated and the job is delivered again with the candidate's choice.

6 Delivery

Each great or normal job becomes one email, created directly in a Gmail label with the IMAP `APPEND` command: it is not sent, does not go through the inbox and triggers no notification. The subject names the resume and the language to apply with:

```
[data · EN] Data Engineer · Acme · Sao Paulo, SP
```

Great jobs go to `JOBS/GREAT`, where the candidate tailors the resume; normal jobs go to `JOBS/READY`, to apply with the base resume named in the subject. Rejected jobs are listed, each with its reason and link, in one summary email per run in `JOBS/SUMMARY`, so a wrong rejection can be spotted and the profile corrected. After the emails are created, `delivered_at` is filled in, so a job is never delivered twice. Every text of these emails can be overridden in the configuration; the author's own instance delivers everything in Portuguese without any change to the code.

7 Orchestration

7.1 Runtime

The runtime is a Docker image based on `python:3.12-slim` with Node.js and Claude Code installed. `run.sh` runs one step in the container; the data folder is mounted at `/data`, and the container runs with the host user's id, so the files it writes belong to the user. Each step accepts `--dry-run`, and most accept `--limit`.

7.2 Scheduled runs

Three GitHub Actions workflows run in the private repository (Table 6). After each run, the pipeline workflow commits the updated data back to the repository, even when a step failed, so whatever was done is kept. A repository variable, `HIRELLME_ENABLED`, must be true for the scheduled workflows to run, so they never run in the public copy of the code.

Workflow	When	What
`pipeline`	04:07 and 13:17 (Brasília), or by hand	The four steps; commits the data
`watchdog`	One hour after each scheduled run	Checks that the run happened and succeeded; if not, starts it again and emails the candidate
`tests`	Every push	The unit tests

Table 6: GitHub Actions workflows.

GitHub may delay or skip scheduled runs under load, which happened in the first days of operation; the watchdog exists for that reason. The scheduled times are set a few minutes away from the full hour, when the scheduler is busiest. A full run uses about two to three minutes of runner time, so the whole setup, including the watchdog and the tests, fits well within the free allowance for private repositories.

7.3 Notices

Notices are emails the pipeline creates in the candidate's inbox: a missed or failed scheduled run (from the watchdog), a triage that needs attention (from the triage exit code) and a Claude token about to expire (30, 7 and 1 day before the one-year validity ends, checked on the first run of each day). Because notices are delivered through Gmail, a revoked Gmail password would also silence them; GitHub's own email for failed workflows covers that case.

8 Reliability

The design rests on three invariants:

- **No loss.** Every job found is appended to the raw layer before its email leaves the queue; every pending job stays pending until it is decided.
- **No duplicates.** Emails are recognised by `Message-ID`, jobs by the hash of their link, decisions by the job id, deliveries by `delivered_at`.
- **Idempotent steps.** Running a step twice, or a whole run after a partial one, gives the same final state.

Failure	Handling
Gmail refuses the login	The run fails with a clear message (revoked app password, or the account password changed); GitHub emails the failure
An Indeed link does not answer	The email stays in the queue and is read again; after two days it goes to `JOBS/MANUAL` with the jobs that were read
Gupy search fails	The rest of the run goes on; the next run searches again from the last successful one
LLM unavailable or out of quota	Jobs wait in the queue (Table 5); the candidate is told if it persists
Claude token near expiry	Notice 30, 7 and 1 day before
Scheduled run skipped by GitHub	The watchdog starts it and sends a notice
Run stopped halfway	The next run continues from the files; nothing is redone or lost

Table 7: Failure modes and how each one is handled.

An incident. On 9 November 2026, a dry run before a deployment found four new Indeed alerts going to `JOBS/MANUAL` with "no job found". A diagnostic that printed only the host, path and parameter names of the links (never their values) showed that Indeed had moved from `engage.indeed.com/f/a/...` links to `/rc/clk/dl?jk=...` links, plus `/pagead/` links for sponsored jobs that answer HTTP 403. The link pattern was generalised to any Indeed domain, sponsored links were kept as they are, and the four alerts produced 80 jobs in the next run. Nothing was lost in between: the alerts had stayed in the queue. The episode shows both the main weakness of the design, its dependence on email formats, and the value of dry runs and of a queue that keeps what it cannot read.

9 Security and Privacy

- **Secrets** (the Gmail app password and the LLM credential) live only in a local `.env` file and in GitHub Secrets. The `.env` file is never committed; in GitHub Actions it is created from the secrets at the start of each run, with permissions restricted to the owner.
- **Login tokens** contained in LinkedIn alert links are never stored, and diagnostics print link structure, not link values.
- **Personal data** (profile, configuration, collected jobs and decisions) is listed explicitly in the public repository's `.gitignore`, each file with a `.example` counterpart; it lives only in a private repository.
- **Billing safety.** The Claude Code provider refuses to run when an API key is present in the environment, so a subscription setup can never fall back to paid usage by mistake.
- **Platforms.** LinkedIn and Indeed are not logged into or crawled: the pipeline reads the alerts they already send to the candidate. Indeed links are resolved by following their redirects without opening the job page, and this can be turned off; the Gupy search sends a few paced requests per run.

10 Results

10.1 First run

The first run processed two weeks of alerts accumulated before the pipeline existed (Table 8).

Stage	Volume
Alert emails read	45 (LinkedIn and Indeed), plus the Gupy search
Raw postings (bronze)	468: LinkedIn 310, Indeed 46, Gupy 112
Unique jobs (silver)	316: 152 duplicates removed (32%), 46 of them the same job across sources
Triage (gold)	316 of 316 valid decisions, in 8 requests of up to 40 jobs
Delivered	33 great, 172 ready to apply, 111 rejected (one summary email)
Duration	About 10 minutes

Table 8: First run, over two weeks of alerts.

A third of the raw postings were duplicates: the same opening sent again by the same platform on later days, or published on more than one platform. About a third of the unique jobs were rejected, mostly for seniority, location or the type of role, which is the reading the pipeline saves.

10.2 Daily runs

After the first run, each run processes only what is new. Table 9 shows a run of 9 November 2026 that read four Indeed alerts, and a run that processed two jobs sent by the candidate.

	Alerts run	Candidate's jobs
Emails read	4	2
Postings collected	85 (80 Indeed, 5 Gupy)	5 (2 sent, 3 Gupy)
New jobs after cleaning	48	3
Requests to the LLM	2 (40 + 8 jobs)	1
Great / normal / rejected	3 / 29 / 16	1 / 1 / 1
Input / output tokens	16,211 / 10,302	4,495 / 1,478
Triage time	46 s	11 s
Total runner time	2 min 40 s	1 min 52 s

Table 9: Two daily runs. In the second one, the two jobs sent by the candidate were delivered as great (tagged) and normal (untagged); the rejected job came from Gupy.

Most of the runner time is spent building the Docker image; collection takes about ten seconds and the triage of 48 jobs under a minute. The output tokens, about 200 per job, are dominated by the one-line reasons.

11 Testing

The 19 unit tests run without internet, Gmail or LLM, on every push. They cover:

- the parsers: LinkedIn blocks, Indeed lists and single-job alerts, offline decoding of compressed links, the new and sponsored Indeed links, links sent by the candidate, subject tags and

fields;

- the cleaning: merges across sources and excluded titles, on temporary data folders;
- the triage: validation of the model's answer, the error kinds of every provider, the rules for jobs sent by the candidate;
- the delivery: email subjects, including jobs delivered without a decision, and the translation of the texts through the configuration;
- the handling of time in UTC.

Beyond the unit tests, every step has a dry run that reads the real sources and reports what it would do, without writing or moving anything; changes are deployed only after a dry run against the real inbox.

12 Reproducibility

```
cp .env.example .env                          # credentials
cp data/config.json.example data/config.json
cp data/profile.md.example data/profile.md
./build_run.sh                                # build the image
./run.sh triage --check                       # does the LLM answer?
./run.sh collect --dry-run                    # what would be collected
./run.sh pipeline                             # collect, clean, triage, deliver
```

For daily runs, the project is copied into a private repository with the data files committed, the secrets are set with the GitHub CLI and the workflow is enabled with the repository variable; the README gives the commands. The raw layer, together with the configuration and the profile, is enough to rebuild the clean table exactly; the gold layer depends on the model's answers, which are not deterministic, and is therefore kept rather than recomputed.

13 Limitations and Future Work

- **Little information in some alerts.** LinkedIn alerts carry only title, company and location, so those jobs are triaged on less evidence than Gupy jobs, which come with full descriptions.
- **Dependence on email formats.** The parsers follow the current alert formats, which change without notice (Section 8). Unknown formats are set aside rather than lost, but need a code change.
- **Unofficial search endpoint.** The Gupy search uses the endpoint of the public portal, not a documented API, and may change.
- **No measured accuracy.** The triage has not been evaluated against labelled data. A first step that needs no manual labelling is to measure the agreement between the small model used daily and a larger one on the same jobs, and to look at the disagreements.
- **One candidate per instance.** Each person needs their own private repository, Gmail setup and profile. A template repository with a setup workflow driven by a form, and a workflow that pulls code updates without touching the data, would let someone without a computer set it up from a phone.
- **Scheduler delays.** GitHub may delay or skip scheduled runs; the watchdog limits the damage to about an hour.

References

[1] M. Crispin. Internet Message Access Protocol – Version 4rev1. RFC 3501, IETF, 2003.

[2] P. Deutsch. GZIP file format specification version 4.3. RFC 1952, IETF, 1996.

[3] S. Josefsson. The Base16, Base32, and Base64 Data Encodings. RFC 4648, IETF, 2006.